

Microservicio de Notificaciones Grupo J

Documentación técnica y modelo entidad relación de la base de datos

Plataforma de comercio electrónico distribuida - Taller Semana 8

Fecha de elaboración: 3 de octubre de 2026

Propósito y alcance

El microservicio recibe eventos de pedidos, pagos, facturación e inventario mediante RabbitMQ, genera mensajes para el cliente y los conserva en SQLite. Ofrece una API REST y una bandeja web para consultar notificaciones y marcarlas como leídas. Este documento describe el modelo realmente implementado, sus relaciones, atributos, restricciones y configuración.

Grupo J administra una única tabla propia, Notificacion. Cliente y Pedido se incluyen en el diagrama como entidades externas de referencia. Sus datos completos pertenecen a los otros servicios. La separación permite procesar eventos sin realizar consultas directas a las bases de los compañeros.

Estado de implementación

La versión local utiliza Node.js, Express, SQLite y RabbitMQ. El proyecto incluye un esquema alternativo para MySQL; no existe un adaptador MySQL activo. Tampoco se implementaron envío SMTP ni autenticación por cliente. Estos límites se distinguen de la funcionalidad verificada para evitar atribuir al proyecto capacidades pendientes.

Contenido

Tecnologías y arquitectura; diagrama entidad relación; diccionario de datos; esquema SQL y migración; eventos y API; puertos, ejecución, pruebas e integración.

Tecnologías y arquitectura

Lenguajes y formatos

JavaScript desarrolla backend, consumidor, frontend y pruebas. HTML5 y CSS3 estructuran y presentan la interfaz. SQL define el almacenamiento. JSON transporta eventos y respuestas HTTP. YAML configura Docker Compose. El frontend utiliza JavaScript nativo, sin React, Angular ni Vue.

Entorno framework y bibliotecas

Node.js 24 o superior ejecuta módulos ES. Express ^5.1.0 gestiona las rutas HTTP y sirve archivos estáticos. amqplib ^0.10.9 conecta productores y consumidores mediante AMQP. jsdom ^29.1.1 es una dependencia de desarrollo para las pruebas funcionales del frontend. package-lock.json conserva las versiones exactas resueltas.

Los módulos integrados incluyen node:sqlite para DatabaseSync; node:crypto para SHA-256; node:fs y node:fs/promises para archivos; node:path y node:url para rutas; node:test y node:assert/strict para pruebas; y node:os para directorios temporales de prueba.

Herramientas

npm instala y ejecuta dependencias. Docker y Docker Compose despliegan la aplicación y RabbitMQ. La imagen de aplicación es node:24-bookworm-slim; la del broker es rabbitmq:4.2-management. RabbitMQ Management permite observar colas, exchanges y conexiones. PowerShell se utiliza para los comandos del entorno Windows.

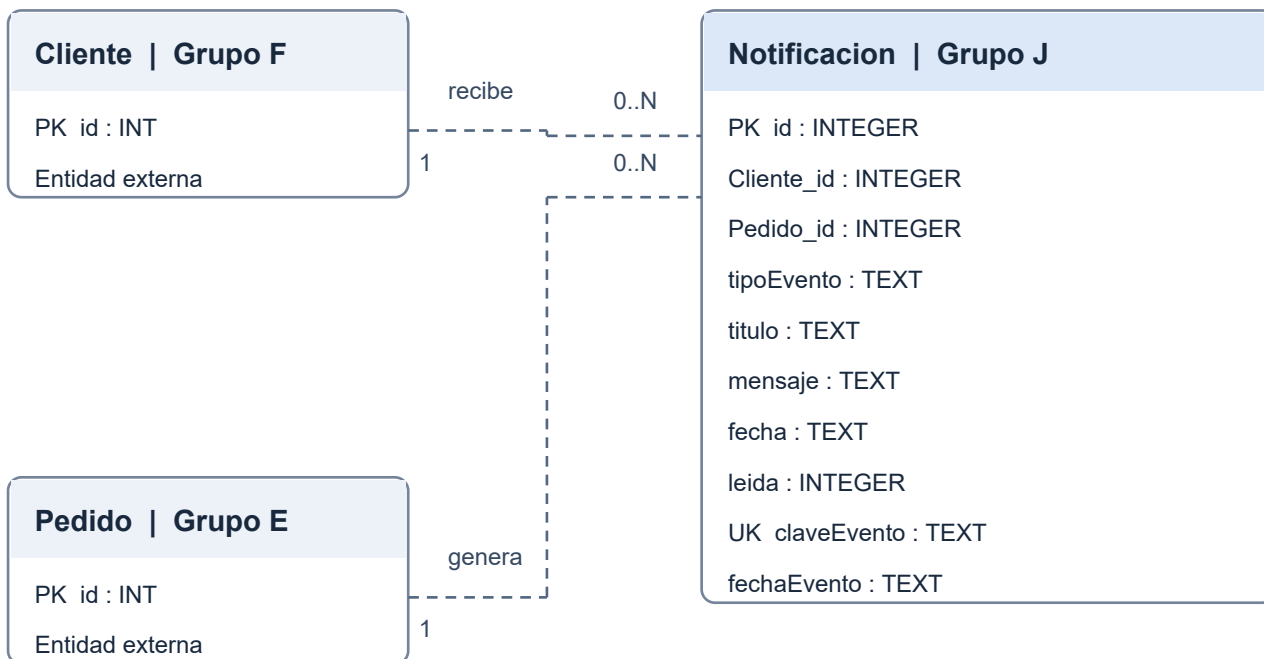
Recorrido de una notificación

```
Productor -> RabbitMQ exchange -> cola de Grupo J
           -> consumidor -> validar evento
           -> generar mensaje -> guardar en SQLite -> ACK
```

```
API REST -> repositorio SQLite -> bandeja web
```

El productor publica al exchange compartido. El consumidor procesa el JSON y confirma únicamente después del guardado o de reconocer una entrega idéntica ya almacenada. Los mensajes inválidos se rechazan hacia una cola de errores; los fallos temporales provocan reconexión y reentrega sin confirmar prematuramente.

Modelo entidad relación



Línea discontinua: relación lógica entre servicios, sin FK física.

PK: clave primaria. UK: restricción única. Tipos del motor SQLite activo.

Cardinalidades

Un Cliente puede recibir cero o muchas notificaciones. Cada Notificacion tiene un destinatario identificado por Cliente_id. Un Pedido puede generar cero o muchas notificaciones y cada Notificacion corresponde a un pedido identificado por Pedido_id. Las cardinalidades expresan el modelo de dominio; SQLite no comprueba la existencia de filas en los servicios externos.

Propiedad de los datos

Cliente pertenece al Grupo F y Pedido al Grupo E. Grupo J no crea esas tablas ni almacena nombres, contraseñas, productos o pagos. Cliente_id corresponde a Cliente.id; Pedido_id corresponde a Pedido.id. Se mantienen referencias lógicas, sin restricciones FOREIGN KEY entre bases de diferentes microservicios.

En el evento y en la respuesta pública los identificadores se llaman clienteld y pedidold. El repositorio los adapta a Cliente_id y Pedido_id, conservando los contratos. No existe una relación física Cliente-Pedido adicional dentro de esta base.

Diccionario de datos y restricciones

id

INTEGER PRIMARY KEY AUTOINCREMENT. Identifica de forma única la notificación. SQLite lo genera al insertar; no tiene que coincidir con el id del pedido. Pueden existir saltos en la secuencia.

Cliente_id y Pedido_id

INTEGER NOT NULL. Identifican al destinatario y a la compra. En bases nuevas existen CHECK para valores positivos; el consumidor también exige enteros positivos seguros. No son claves foráneas físicas.

tipoEvento

TEXT NOT NULL. Conserva el nombre del evento. El consumidor admite PedidoCreado, PagoAprobado, PagoRechazado, FacturaGenerada e InventarioInsuficiente. La lista se valida en la aplicación, no mediante un CHECK SQL.

titulo y mensaje

TEXT NOT NULL. Contienen el texto generado para el usuario. Se obtienen de las plantillas del servicio; no se calculan al consultar la bandeja.

fecha

TEXT NOT NULL. Guarda la fecha de creación de la notificación en ISO UTC, generada al persistirla. No representa necesariamente el momento en que ocurrió el evento.

leida

INTEGER NOT NULL DEFAULT 0, con CHECK para 0 o 1. Cero indica sin leer; uno indica leída. La API lo convierte a booleano y el PATCH actualiza el valor a 1 de forma idempotente.

claveEvento

TEXT NOT NULL UNIQUE. Almacena el SHA-256 del JSON canónico. Evita guardar dos veces un payload idéntico, incluso si cambia el orden de propiedades. No se expone en la API.

fechaEvento

TEXT NOT NULL. Conserva la fecha ISO recibida, con o sin zona según el contrato. No se expone en la respuesta pública.

Todos los campos son obligatorios. No se almacena fecha de lectura, correo, estado de envío ni un historial de cambios de lectura; esos atributos no forman parte del modelo implementado.

Esquema SQL persistencia y migración

Esquema SQLite de una base nueva

```
CREATE TABLE IF NOT EXISTS Notificacion (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  Cliente_id INTEGER NOT NULL CHECK(Cliente_id > 0),  
  Pedido_id INTEGER NOT NULL CHECK(Pedido_id > 0),  
  tipoEvento TEXT NOT NULL,  
  titulo TEXT NOT NULL,  
  mensaje TEXT NOT NULL,  
  fecha TEXT NOT NULL,  
  leida INTEGER NOT NULL DEFAULT 0 CHECK(leida IN (0,1)),  
  claveEvento TEXT NOT NULL UNIQUE,  
  fechaEvento TEXT NOT NULL  
);  
CREATE INDEX IF NOT EXISTS idx_cliente  
  ON Notificacion(Cliente_id, id);  
CREATE INDEX IF NOT EXISTS idx_notificacion_pedido  
  ON Notificacion(Pedido_id, id);
```

Guardado consultas e índices

El repositorio usa consultas parametrizadas y un INSERT con ON CONFLICT(claveEvento) DO NOTHING. Después obtiene la fila por la clave única. Los índices facilitan búsquedas por cliente y trazabilidad por pedido. Las listas se ordenan por id descendente. WAL y busy_timeout=5000 configuran el manejo de acceso a SQLite.

Migración del modelo anterior

Si existe notificaciones, el repositorio la renombra a Notificacion y cambia clienteld a Cliente_id y pedidold a Pedido_id dentro de una transacción. Conserva filas, ids, fechas, lectura y claves. Al renombrar una tabla anterior no se incorporan retroactivamente los nuevos CHECK de ids positivos; la aplicación sigue validando los eventos. No se eliminan registros.

Ubicación y alternativa MySQL

Localmente se utiliza ./data/notificaciones.sqlite; en Docker, /app/data/notificaciones.sqlite dentro del volumen notificaciones_data. SQLite no requiere puerto ni servidor externo.

database/mysql/001_notificacion.sql define un esquema InnoDB con id BIGINT UNSIGNED, textos VARCHAR/TEXT, leida TINYINT e índices equivalentes en una base propia de J. Ejecutar ese archivo no cambia el motor de la aplicación.

Eventos API y ejemplo de datos

Contratos de entrada

PedidoCreado conserva el contrato de Semana8. Los otros cuatro eventos utilizan como supuesto sugerido evento, pedidoId, clienteId y fecha, pendiente de acuerdo con los productores. No se modifican unilateralmente los contratos del equipo.

```
{
  "evento": "PedidoCreado",
  "pedidoId": 1001,
  "clienteId": 25,
  "fecha": "2026-10-01T10:30:00",
  "total": 5000000,
  "productos": [
    {"productoId": 15, "cantidad": 2, "precio": 2500000}
  ]
}
```

Grupo J no guarda total ni productos. productos[].productId representa Producto.id, no Inventario.id. Esos datos se validan en PedidoCreado, pero su administración pertenece a otros servicios. Los identificadores del ejemplo no demuestran registros reales en sus bases.

Resultado y acceso

El ejemplo crea una notificación para Cliente_id=25 y Pedido_id=1001, con tipoEvento PedidoCreado, título Pedido recibido, mensaje dirigido al usuario y leida=0. La respuesta pública contiene id, clienteId, pedidoId, tipoEvento, título, mensaje, fecha y leida; oculta claveEvento y fechaEvento.

```
GET    /api/notificaciones
GET    /api/notificaciones/{id}
GET    /api/notificaciones/cliente/{clienteId}
PATCH /api/notificaciones/{id}/leida
GET    /health/live
GET    /health/ready
```

Las listas admiten limit y offset: 100 por defecto y máximo 500. Se devuelve 400 ante parámetros inválidos, 404 para recursos inexistentes y 500 ante errores internos. Readiness devuelve 503 si RabbitMQ está desconectado. La bandeja filtra por evento, estado y cliente; sus contadores se calculan sobre la página cargada. Las consultas evitan caché y la actualización solicitada durante un marcado espera al guardado.

Configuración ejecución y verificación

Puertos y variables

HTTP usa 3000 dentro del contenedor y 3010 en el computador de trabajo. El Compose local y .env.example usan 3000 externo por defecto; el Compose de integración usa 3010. HOST_PORT cambia el puerto externo y PORT el de Node local. AMQP utiliza 5672 y RabbitMQ Management 15672. Los puertos se publican en 127.0.0.1; SQLite no utiliza puerto de red.

Las variables restantes son DATABASE_PATH, RABBITMQ_URL, RABBITMQ_EXCHANGE, RABBITMQ_EXCHANGE_TYPE, RABBITMQ_QUEUE, RABBITMQ_BINDINGS, RABBITMQ_PREFETCH y RABBITMQ_RECONNECT_MS. RABBITMQ_PASSWORD personaliza el broker local. El archivo configuraciones.txt documenta estos valores. Los .env reales no se incluyen en la entrega.

RabbitMQ y ejecución

Valores sugeridos: exchange comercio.eventos, tipo topic; cola grupo-j.notificaciones; routing keys iguales a los cinco eventos. El exchange de errores es grupo-j.notificaciones.errores.exchange y la DLQ grupo-j.notificaciones.errores. Prefetch es 10 y la reconexión espera 5000 ms. Cada grupo consumidor debe tener su propia cola.

```
docker compose up -d --build
# HOST_PORT=3010 para http://localhost:3010/

npm ci
npm start
npm test
npm run test:integration
npm run publish -- examples/PedidoCreado.json
```

Evidencia y límites

En la revisión previa del 3 de octubre aprobaron 16 pruebas locales. La integración real con RabbitMQ aprobó en la revisión anterior: eventos, HTTP, frontend, DLQ, duplicados y recuperación. Se verificó conservación de cinco filas al recrear el contenedor y se comprobó el marcado persistente al actualizar y recargar. Son resultados del entorno probado, no una garantía universal del ecosistema.

La integración con los productores reales, los contratos pendientes y cualquier conexión a MySQL requieren coordinación del equipo. No se implementaron correo SMTP ni autenticación por cliente. La prueba DOM no sustituye la inspección visual del frontend. La publicación directa a GitHub quedó pendiente de permiso de escritura; no se afirma que el repositorio remoto ya contenga la entrega.

Fuentes del documento

Código local: src/repositories/notificationRepository.js, src/models/event.js, src/app.js, public/app.js, tests y database/mysql/001_notificacion.sql. Configuración: package.json, package-lock.json, configuraciones.txt y los dos Compose. Enunciado: Semana8.docx y modelo de datos aportados para el proyecto.